TERRSVUSION v3

Memoria KV paginada, persistente y auditable

con autoridad BF16 exacta y decodificación multitoken verificable

Informe técnico verificado - 10 páginas

Versión 1.0 | Proyecto Terrsvusion | 29 de octubre de 2026

Resumen

Terrsvusion es un motor experimental en Rust para MiniCPM5-1B que mantiene los pesos y la autoridad numérica en BF16, pero desacopla la residencia del KV de su direccionamiento. Los estados históricos se sellan en un almacenamiento de solo anexado (append-only) verificable; un catálogo dimensional compacto de cuatro bits, firmas SimHash y micrometadatos sirven exclusivamente para navegar. La ruta actual combina prefijos persistentes, recuperación exacta, verificación multitoken por consulta y reversión provisional.

Resultado verificable

48 pruebas correctas, Clippy estricto limpio y compilación de producción completa en un directorio aislado. La omisión certificada de estados a tolerancia 1e-6 sigue sin demostrarse; la aceleración observada procede hoy de reutilizar prefijos y amortizar varios tokens sin cambiar el KV BF16 final.

Palabras clave: KV cache, BF16, paginación, almacenamiento frío, prefijos y decodificación especulativa.

Fuente primaria: código ejecutable verificado y métricas reproducibles identificadas.

1. El problema que Terrsvusion intenta resolver

En un Transformer autorregresivo, el prefill transforma todos los tokens del prompt en estados K/V y el decode genera un token cada vez consultando el histórico. El KV crece linealmente con la longitud, mientras la atención causal del primer prefill puede crecer aproximadamente de forma cuadrática. La tensión central es conservar información recuperable sin mantener todo el histórico caliente ni introducir una segunda verdad numérica.

Contrato de diseño

- Pesos originales y cálculo del modelo en BF16. No se crea un archivo de pesos Q4.
- Los cuatro bits son una señal de direccionamiento: reducen el catálogo de búsqueda, no reemplazan K/V.
- El almacenamiento frío conserva K/V BF16 y cada recuperación relevante verifica integridad.
- Un selector puede priorizar regiones, pero solo el certificado puede autorizar una omisión.
- La paridad fuerte se comprueba con ids generados y con un SHA-256 del KV lógico completo.

Comparación conceptual



La distincion esencial

Terrsvusion no promete que cuatro bits contengan toda la semántica del KV. Los cuatro bits indican dónde buscar. La información autoritativa continua en BF16 y puede reconstruirse desde el almacenamiento frío.

Fórmula de memoria de referencia

Para N tokens, L capas, Hkv cabezas, dimensión d y b bytes por escalar, el KV denso ocupa: $M_{KV} = N \times L \times 2 \times Hkv \times d \times b$. Con la geometría probada (24 capas, 2 cabezas KV, d=128, b=2), cada token añade 24.576 bytes. Terrsvusion desplaza la mayor parte al almacenamiento de respaldo y mantiene residentes catálogos, certificados, colas y metadatos.

Alcance honesto

Reducir la residencia estable del KV no equivale a reducir el RSS total del proceso: pesos, allocator de LibTorch, temporales SDPA y buffers del sistema siguen existiendo.

2. Arquitectura canónica y fronteras de autoridad

El crate expone una biblioteca y tres binarios: servidor HTTP, chat local y banco de pruebas de contexto largo. La ruta de servicio real pasa por una única autoridad de modelo y una cola acotada; los trabajadores de conversión preparan registros, pero no deciden tokens.



Mapa de módulos ejecutables

Modulo	Responsabilidad real	No es autoridad de
torch_runtime.rs	Modelo, KV, atención, prefijos, decode	La integridad del disco por sí sola
adaptive_extent.rs	Estados variables, 4-bit, LSH, grafo	Omitir K/V
attention_certificate.rs	Cotas y selección por masa exacta	Logits finales universales
cold_store.rs	Solo anexado, hashes, lectura parcial	Relevancia semántica
conversion_worker.rs	Preparacion paralela y orden de commit	Elegir tokens
inference_worker.rs	Propietario del modelo y backpressure	Paralelismo de una autoridad
shadow_metrics.rs	Diagnóstico escalar sin I/O adicional	Cambiar selección

Separacion de planos

Plano numérico: MiniCPM5, logits y K/V BF16. Plano de navegación: estados dimensionales, firmas, LSH y micrometadatos. Plano de persistencia: ColdStore y manifiestos de prefijo. Plano operativo: Axum/Tokio, cola, trabajadores y métricas. Esta separación evita que una heurística de búsqueda se convierta accidentalmente en autoridad sobre la respuesta.

> **Invariante de seguridad**
> Un fallo de LSH devuelve cero semillas y cede el control al camino certificado. No existe una ruta de respaldo que interprete el fallo de búsqueda como permiso para producir una respuesta aproximada.

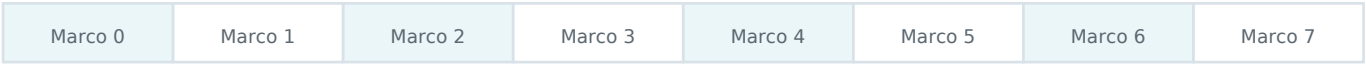
3. Paginación lógica y micrometadatos dimensionales

Terrsvusion separa las regiones logicas de contenido de los marcos físicos de almacenamiento. Un estado dimensional se abre y cierra según sorpresa respecto a su centroide y cambio temporal. Por ello no existe una longitud semántica fija de 64, 128 o 512 tokens: esos valores pertenecen al I/O físico, no al significado.

Eje de tokens



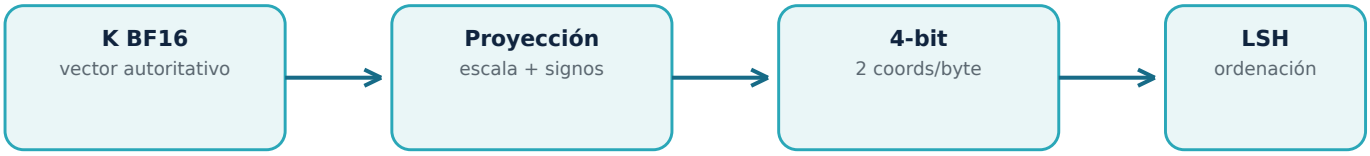
Límites lógicos variables definidos por contenido



Marcos físicos uniformes para lectura y commit

Huella dimensional de cuatro bits

Cada token recibe coordenadas firmadas en el intervalo [-7, 7], dos por byte, y una escala local. El objetivo no es reconstruir el vector K desde esos digitos, sino preservar suficiente geometría relativa para ordenar vecinos. El índice SimHash usa cuatro tablas, proyecciones deterministas, bits direccionales y bandas de magnitud y dispersión.



Micrometadatos comunicantes

Los estados almacenan deltas compactos y relaciones que permiten que una señal relevante se propague por la malla lógica. El grafo no añade todas las regiones relacionadas a la carga inicial; solo desempata el orden de exploración. Esta regla evita que una conectividad densa destruya la ventaja de la paginación.

- LSH propone candidatos; no garantiza cobertura ni exactitud.
- El estado reciente se conserva siempre como ancla causal.
- Las coordenadas token permiten volver desde una región a los bytes BF16 exactos.
- Selección lógica y marcos físicos pueden cruzarse sin perder reversibilidad.

> **Lo que no debe afirmarse**
>
> Los micrometadatos no son una memoria semántica autosuficiente, no sustituyen el prefill inicial y no contienen el libro entero en cuatro bits.

4. ColdStore: almacenamiento BF16 recuperable

ColdStore es un archivo de solo anexado (append-only) con registros comprometidos. Cada página conserva geometría, offsets, K, V, checksums rápidos, hashes por token, digest SHA-256 y un marcador final de commit. Al reabrir, una cola final incompleta no entra en el catálogo. Las lecturas parciales verifican los tokens recuperados antes de entregarlos a la atención.



Registro lógico simplificado. Los offsets reales se validan antes de leer.

Flujo de commit y recuperación



Identidad y pruebas de corrupción

- FNV-1a y checksums permiten rechazo rápido de alteraciones locales.
- SHA-256 por registro detecta corrupción aunque un checksum rápido fuera recompuesto.
- El digest KV lógico recorre [capa][K/V][cabeza][token][dimensión], independiente de la fragmentacion física.
- Una lectura diagnóstica del digest se excluye de los contadores operativos para no fingir coste de decode.

Prefijos persistentes direccionados por contenido

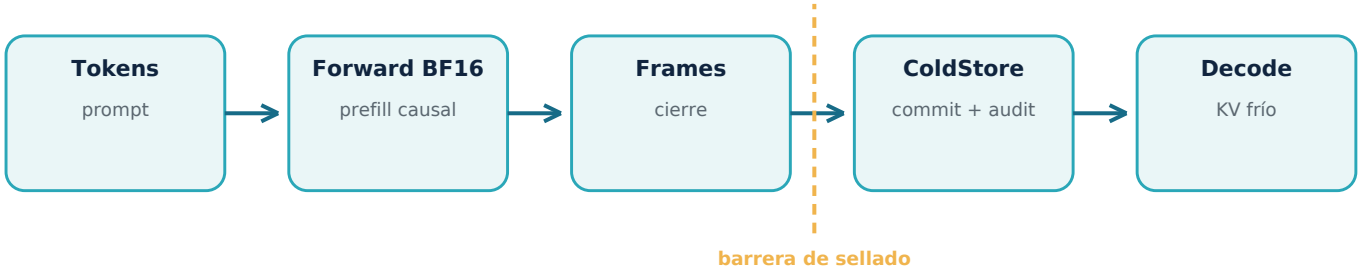
La clave de prefijo incorpora identidad de modelo, tokenizador, IDs exactos y geometría. La publicación usa una zona de preparación y un renombrado atómico. En un acierto, el runtime copia el respaldo a una zona de trabajo, valida manifiesto y registros, reconstruye el eje dimensional y rehidrata el BF16 necesario para procesar el sufijo. El prefijo publicado nunca se modifica durante la decodificación.

> **Garantía precisa**
>
> La persistencia elimina el cómputo Transformer repetido del prefijo. Todavía paga búsqueda, copia y rehidratación; por tanto, un acierto no significa I/O cero ni memoria cero.

Esta arquitectura convierte el contexto conocido en un artefacto recuperable y auditable. Su valor principal es evitar repetir trabajo ya certificado, no prometer que la primera ingestión de un documento desaparezca.

5. Prefill, barrera fría y reutilización

El prefill inicial sigue siendo atención causal BF16 completa. Terrsvusion procesa el prompt por chunks para controlar temporales, prepara marcos cerrados en un worker y, al final, sella colas, drena conversiones, audita el almacenamiento y libera el conjunto temporal. Esto reduce la residencia posterior, pero no elimina el coste matemático de leer por primera vez el documento.



Dos caminos de una consulta

FALLO de prefijo

1. Prefill del prefijo
2. Sellado + persistencia
3. Rehidratación

ACIERTO de prefijo

1. Búsqueda por SHA-256
2. Auditoría + copia de trabajo
3. Rehidratación BF16

Medición real documentada

Evento	Tokens	Prefill	Lookup	Rehidratacion
Primera pregunta: fallo	5.454	15,954 s	0,00002 s	0,155 s
Segunda pregunta: acierto	5.454	0,549 s	0,654 s	0,147 s
Acierto tras reinicio	5.454	0,443 s	0,701 s	0,163 s

En esta corrida, el acierto redujo el prefill medido aproximadamente un 96,6 %. La recuperación tras reinicio demuestra persistencia entre procesos. No permite extrapolar velocidad universal: la decodificación de 16 tokens siguió dominando con unos 22,5 s en aquella configuración.

Interpretación correcta

La mejora fuerte aparece cuando muchas preguntas comparten un documento estable. Para un único mensaje sin frontera documento-pregunta, el runtime no puede inferir de forma segura que parte debe persistir.

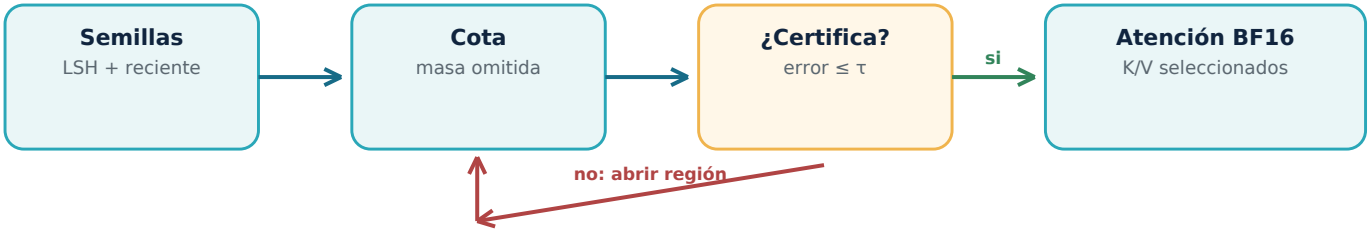
Siguiente frontera técnica: evitar la copia y rehidratación completas mediante un almacenamiento compartido inmutable y direccionable, conservando la misma validación criptográfica.

6. Branch-and-bound y certificado de masa

Durante decode, el selector intenta abrir primero las regiones más prometedoras. Para cada cabeza estima una cota superior de score QK para las regiones cerradas. Si la masa softmax omitida, multiplicada por la magnitud máxima de V, queda bajo la tolerancia, la omisión puede certificarse; en caso contrario se abre otra región.

Cota local usada

$||\Delta Y||_\infty \leq 2 \times \alpha_{omitida} \times \max_t ||V_t||_\infty$. La masa α y la magnitud de V se mantienen separadas. La tolerancia vigente es $\tau = 1e\text{-}6$ por evento local.



Por que hoy no evita estados a 1e-6

Las cajas AABB son seguras pero laxas: combinan extremos de dimensiones pertenecientes a tokens distintos y construyen un máximo virtual. Sin embargo, el oraculo exacto demostró que la geometría no es el único bloqueo. Incluso con scores exactos, la cota basada en max|V| necesita una tolerancia mínima observada de 2,783e-6 para omitir un solo token en la prueba 1.024/8.

Tolerancia	Tokens omitibles	% de 172.704 oportunidades
1e-6	0	0 %
1e-5	0	0 %
1e-4	5	0,0029 %
1e-3	441	0,2554 %

La instrumentacion shadow observo holguras AABB elevadas y una sobreestimacion local de la cota V de aproximadamente 2-5x en cabezas con omisión experimental. No propaga esos estados alternativos hasta logits; por tanto mide conservadurismo local, no seguridad end-to-end.

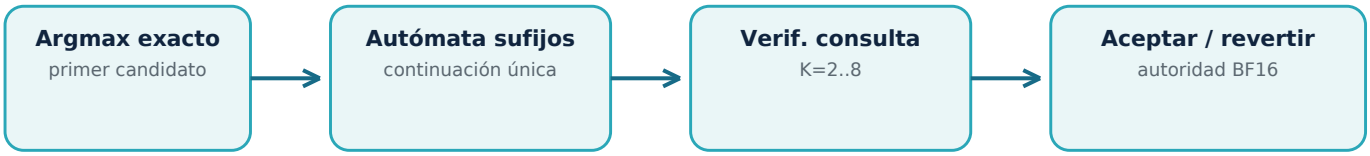
Resultado negativo valioso

hierarchy_k_tokens_avoided = 0 no significa que el sistema haya perdido datos. Significa que, con el contrato actual, se negó correctamente a descartar estados que no podía certificar.

La mejora futura rigurosa debe estrechar simultáneamente las cotas K y V o derivar una garantía ligada a logits. Subir τ para forzar artificialmente una cifra positiva cambiaría el contrato y no sería una corrección.

7. Decodificación exacta, especulación y reversión

El avance principal de la ruta actual no es omitir KV, sino amortizar varios tokens por pasada sin aceptar la divergencia numérica de la ejecución en bloque convencional. Una pasada multitoken GEMM fue rápida, pero generó diferencias de hasta 0,25 en logits y un SHA-256 del KV distinto. Terrsvusion adoptó una ruta por consulta: comparte proyecciones y MLP en bloque, pero ejecuta cada atención con longitud uno y su prefijo causal exacto.



Proponente determinista y política adaptativa

Un autómata de sufijos, construido en tiempo lineal sobre el prompt, extiende la propuesta solo cuando el estado observado tiene una única transición y no es terminal. Esa unicidad documental no obliga al modelo: cada candidato se verifica. K comienza en 2, puede duplicarse hasta 8 tras aceptación completa y vuelve a 2 con un período de enfriamiento proporcional tras un rechazo.

Transacción provisional

- begin_multitoken_verification impide confundir un bloque candidato con prefill.
- Los K/V candidatos permanecen en colas no selladas y no llegan a trabajadores ni al almacenamiento de respaldo.
- Si una propuesta falla, rollback_unsealed_to conserva solo el prefijo aceptado.
- La reversión falla en modo cerrado si intentara atravesar almacenamiento ya comprometido.
- logical_state_digest confirma la igualdad del KV final en orden lógico, no físico.

Por qué importa la verificación por consulta

Ruta	K=2	K=4	K=8	KV final
Batch causal convencional	2,07x	4,04x	7,14x	Divergente
Por consulta exacta	1,70x	2,62x	3,21x	SHA idéntico

La diferencia procede del orden de acumulacion numérica entre kernels GEMV/GEMM, no de una carrera. En el diagnóstico, la capa 0 coincidio y la divergencia aparecio desde la capa 1 en 23 de 24 capas. Por eso el batch rápido no fue promovido como autoridad.

Aceleración con contrato fuerte

La ruta por consulta sacrifica parte de la aceleración teórica para mantener error de logits cero, IDs idénticos y SHA-256 del KV idéntico frente a la decodificación secuencial probada.

8. Evidencia experimental y lectura de resultados

Las cifras siguientes son corridas individuales con MiniCPM5-1B, contexto de 1.024 tokens, chunks y marcos de 256. Sirven como evidencia de funcionamiento y paridad en esa geometría; no son una distribución estadística ni una afirmacion universal para otros modelos o hardware.

Ruta canónica, ocho tokens de salida

Métrica	Terrsvusion actual	BF16 denso	Línea base secuencial
Decodificación	0,459 s	0,616 s	1,281 s
Propuestas / aceptadas	8 / 7	-	-
Pasadas objetivo	3	secuenciales	secuenciales
Eventos certificador	0	no aplica	168
Lecturas frías	576	no aplica	1.680
Paridad IDs + KV	sí + sí	referencia	sí

En esta corrida, Terrsvusion fue aproximadamente 1,34× más rápido que el denso en decodificación y 2,79× respecto a su antiguo camino secuencial. El resultado surge de reducir pasadas, eventos del certificador y materializaciones repetidas; no de hierarchy_k_tokens_avoided, que permanece en cero en la ruta certificada tradicional.

Caso adversarial con rechazo

Métrica	Resultado
Tokens finales	10
Propuestos / aceptados	14 / 8
Rondas rechazadas	1
Decodificación Terrsvusion / denso	0,904 s / 0,936 s
Paridad IDs y SHA-256 KV	verdadera

El caso adversarial demuestra que la ruta puede descartar candidatos provisionales sin contaminar el respaldo y aun conservar el estado final. La ganancia fue pequeña; una tasa de rechazo peor puede hacer Terrsvusion más lento que el denso.

Metodología y reproducibilidad

- Validación actual: Ryzen 9 7940HS (8C/16T), 58 GiB RAM, Ubuntu 24.04.4; Rust/Cargo 1.96.0; PyTorch 2.5.1 CPU (8+8 hilos).
- Modelo MiniCPM5-1B BF16: SHA-256 7ab8fd86563125929be78aeec8cb3969c7ed2ead3be1ab9d3ec0a9fa69c8660d.
- Tokenizador: SHA-256 3e065a558a034185fe299917b398685c1facd0169a9eea1e629eb30c171fed81.
- Árbol fuente auditado: SHA-256 agregado a9edeb158e6c34630b4ff4cfd63e41269d26ef8ddfc389ac52973c971b8d217a.
- Target aislado; 48/48 pruebas; formato, Clippy -D warnings y release de los tres ejecutables correctos.
- Las latencias de esta página proceden de corridas individuales registradas; no se reejecutaron durante la edición y no representan p50/p95.

> **Cómo leer el rendimiento**
>
> TTFT, decodificación, RSS, KV residente, bytes físicos, I/O frío y calidad son ejes distintos. Una sola cifra no representa la velocidad total.

9. Conclusiones, límites y trabajo futuro

Que se ha creado realmente

Terrsvusion v3 es un motor Rust de memoria KV virtualizada: conserva la autoridad BF16, desplaza estados sellados a almacenamiento verificable, utiliza un plano compacto de navegación y reutiliza prefijos persistentes. Su decodificación canónica añade verificación multitoken exacta, un proponente documental determinista, política adaptativa y reversión transaccional. La combinación es técnicamente diferenciada, aunque sus piezas tengan precedentes en paginación KV, caché de prefijos, recuperación y decodificación especulativa.

Garantías demostradas

- Los pesos no se cuantizan a Q4; los cuatro bits son metadatos de enrutamiento.
- Las páginas K/V comprometidas se recuperan como BF16 y se verifican.
- Los prefijos sobreviven reinicios y evitan repetir su cómputo Transformer.
- La ruta por consulta probada conserva logits, IDs y digest KV frente al secuencial.
- La reversión no atraviesa páginas selladas ni deja candidatos en el respaldo.

No demostrado o pendiente

- Omitir estados reales bajo τ=1e-6: hierarchy_k_tokens_avoided sigue en cero.
- Reducir la complejidad del primer prefill de un documento desconocido.
- Paridad universal en modelos, CPUs, longitudes y consultas no ensayadas.
- Concurrencia productiva de varias inferencias sobre una única autoridad del modelo.
- Tiempos límite HTTP, cancelación cooperativa, endpoint /ready y supervisión de reinicio.

Hoja de ruta priorizada

Prioridad	Intervención	Criterio de éxito falsable
P0	Banco estadístico y perfiles	p50/p95, energía, RSS e IDs
P1	Cota V por dimensión/subespacio	tokens omitidos >0 sin violaciones
P2	Respaldo compartido de prefijos	acierto sin copia completa, mismo SHA
P3	Proponente más preciso	más aceptación y menor p95 adversarial
P4	Servidor operativo	cancelación, límites, /ready, supervisor

Veredicto

El valor actual no es haber eliminado automáticamente el prefill ni comprimido toda la autoridad a cuatro bits. Es haber construido una ruta reversible, auditable y medible que reduce residencia y trabajo repetido, y acelera la decodificación en pruebas concretas sin cambiar la autoridad BF16.

Referencias y fuente de verdad

[S1] Fuente primaria: terravision-v3/src/{torch_runtime,adaptive_extent,attention_certificate,cold_store,conversion_worker,inference_worker,shadow_metrics,prefill_tch}.rs y README.md, estado 29-10-2026. [1] Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023, pp. 611-626. DOI: 10.1145/3600006.3613165. [2] Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; Ré, C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 35, 2022. arXiv:2205.14135. [3] Leviathan, Y.; Kalman, M.; Matias, Y. Fast Inference from Transformers via Speculative Decoding. ICML, PMLR 202, pp. 19274-19286, 2023. arXiv:2211.17192. [4] Reid, M. et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530, 2024.